

Guidelines

The proposal consists of two parts, the first being a description of the project, and the second being a justification of the project. The total length for both sections combined should be about 500 words at most.

The **description** should provide a high-level overview of the project, compare it to any software programs already in existence (if applicable), list its most important features, and discuss the hardware and software it requires. The description should also indicate who the end user of your system would be, and, if relevant, how it would be maintained. A more detailed description of what you plan to accomplish will be included in Section 5 of the SDF, the Requirements Specification section, so only the top-level overview of the project should be included here.

The **justification** should explain why your team's choice of project is appropriate for this class. The following criteria will be used to judge the appropriateness of the project, and should be considered in writing the justification:

1. The project applies and demonstrates what team members have learned in other classes
2. The project gives each team member the opportunity to extend what s/he has learned in other classes
3. The technical difficulty of the project is appropriate (not too easy, not too difficult)
4. One semester is a reasonable amount of time to develop the project
5. The team as a whole possesses the tools required to complete the project
6. The project is interesting to each teammate, classmates, and the instructor

It is in your best interest to obtain approval for the project before the team gets too far underway, and before the team presents its proposal to the class.

Outline (fill out)

Description

- High level overview
 - Problem
 - Currently no application that couples a molecular dynamics engine to a visualizer
 - Must be performed on a workstation with no internet connection due to security reasons
 - The visualizer's data and the retrieval of protein structures from the protein database must work as designed without a network connection
 - Additional requested improvements from Dondi and Dr. Alavi will be implemented, and functionality will need to be expanded.
 - Solution
 - Create an all-inclusive software for the LMU machine that works as both a molecular dynamics engine and a visualizer
 - Allow a staging device to authenticate and fetch data for the LMU machine
 - Create and format data outputs for the simulations
 - Fulfill faculty functionality requests as they are provided.
 - Where we come in
 - We will create the software, including UI/UX, the simulation engine, the data service, and the visualizer, entirely on the LMU machine
 - A fully contained application with all data resolved from a staging device, which will be performed in advance.
- Compare it to currently existing software programs
 - Currently existing molecular engine simulation tools
 - GROMACS
 - AMBER
 - CHARMM
 - Desmond
 - Currently existing data visualization tools
 - VMD
 - ChimeraX
- Most important features
 - Neon DB: Authentication, user's simulations
 - LMU's machine: script to fetch and queue simulations to be run. Output files are sent to the user's email? Or stored in the database and associated with the experiment under the user's account?
 - Visualization: Molecule visualization, simulation results visualization.
 - Possible additions:
 - *From Dondi:* grouping simulations into experiments, users can create entire experiments which automatically run simulations, changing parameters based on some function (linear, parabolic, etc.) over each simulation - will need to talk to Dr. Alavi to find out what the most useful of

these would be; also allowing users to create their own set of simulations, manually picking different values for one or more parameters, or maybe creating their own function for one or more parameters

- *From Dr. Alavi:* Creating and training an AI agent for molecular dynamics tasks (used like a chatbot inside the web app) for use throughout the web app - (e.g. "create a simulation for protein x with parameters a=1, b=2, c=3...")
- *From Dr. Alavi:* Adding more options to choose from in the pre-existing simulation script creation GUI (adding co-solvents into the water, or adding "pulling" forces to the simulation to see how proteins fold or unfold)
- Required hardware and software
 - Hardware: LMU's powerful machine that actually runs the scripts, personal computers for developing and testing code, staging device for fetching data
 - Software: VS Code or other code editor, github for version control
- End user
 - Dr. Alavi
 - Molecular Physics professionals
- Later maintenance after the project is complete?

Justification

1. The project applies and demonstrates what team members have learned in other classes
2. The project gives each team member the opportunity to extend what s/he has learned in other classes
3. The technical difficulty of the project is appropriate (not too easy, not too difficult)
4. One semester is a reasonable amount of time to develop the project
5. The team as a whole possesses the tools required to complete the project
6. The project is interesting to each teammate, classmates, and the instructor

Final Proposal

This project seeks to create an all-inclusive software for Dr. Alavi's Molecular Physics Dynamics research that allows a local staging device to run simulations from users of an external web app and meet the user experience needs of her team.

The problem is users must run Gromacs, the desired software for physics simulation, on their own devices, with command-line parameters, only assisted by a limited web app. Users must use multiple devices and applications, causing a disjointed experience. Dr. Alavi has a multi-GPU machine with very powerful hardware that they want to handle all the simulations but needs both accessibility and security; the computer is unable to access the general internet.

With MD-Gooey, we aim to implement a three-layer solution: an expanded frontend to take requests, a database to store outputs and to authenticate requests made by the local machine, and lastly, software on the local machine that runs the queried simulations.

Other than this, we will be expanding functionality and adding UX features based on the specifications provided by the stakeholders; currently, but not limited to: expanded GUI functionality and an AI chatbot to create requests with natural language.

This project encapsulates multiple facets of computer science education for each member of the team, from networking and databases to web application development and computer graphics. Each member of our team provides a unique advantage in this project that aligns with their learning and career goals, consisting of:

- Game Design
- UI Design
- Designing for Customers
- Consulting
- Prior Involvement with the Project
- Graphic Design
- API Design and Usage
- Quality Assurance Experience
- IT and End-User Security

The development of a real-life application that encapsulates all of the above interests and skills with actual viable stakeholders is the perfect elevation of and send-off to the university experience.

The scale of the project is iterative, with space to expand or reduce features/functionality to time and customer expectations.

We have already spoken with Dr. Dionisio to verify that the scope of our project aligns the current and future needs of Dr. Alavi with the workload expectations of a five-member senior capstone project.

This project's initial inception was as a simple tool for Dr. Alavi to create the parameters for her molecular dynamics engine, but given the capabilities of development under a 5-member capstone project, the needs and functionality of her program have been expanded accordingly. Our group will make an effort to increase her and any biological physics professionals' user experience while using the associated applications and workstations that our project seeks to improve.

The project takes place on campus, where we have access to all steps in the layered design to create the suite.

Notes

AGILE

Implement Neon DB, possibly some basic frontend for testing
Test that communication between database

Proposal

Dondi Questions

Hi Professor,

Sorry to bombard you with a ton of questions, but I just wanted to clarify some aspects of the project for myself and my group. I know we spoke about a lot of these things in the past, but I wanted to see if you were able to come to a conclusion on any more of the specifics of the project, and also make sure my group fully understands what you, Dr. Alavi, and the ITs department expect from us.

1. What are the LMU machine's specs?
2. Script on the local machine
 - a. How might we set up LMU's local machine to access simulations-to-be-run from the database?
 - b. Will we have to design a program (run locally) to request and then execute the simulations with GROMACS, and capture the output files to be sent back to the database?
3. Database x local machine connection
 - a. Specific strategy to connect local machine to neon database- does machine make requests and get sent files?
 - b. Does it simply read the database with an API?
 - c. Should we design around giving the database itself queueing functionality, or leave queueing functionality for the local device?
4. Frontend ownership
 - a. As the primary frontend developer for MD-Gooey is Lauren, should my group and I expect to also spend time updating the frontend to accommodate the new functionalities we plan to implement (visualization, "my simulations", maybe even a chatbot frontend)?
 - b. Should we not touch the frontend (only change it for testing and demo I guess?), and possibly instead establish regular communication with Lauren with regards to our project and any frontend changes we may need her to take care of?
5. Ownership
 - a. Which of the above aspects of the project do you expect us to take ownership of designing (as opposed to the aspects of the plan you already ran by ITS).
6. If some of these questions are better asked of the ITs department, the person I should email is Brian Drawert, right?